REPUBLIQUE DU CAMEROUN
Paix-travail-patrie
****
UNIVERSITE DE YAOUNDE 1
****
ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE DE YAOUNDE
****
AN-ING 1

REPUBLIC OF CAMEROON
Peace-work-fatherland
****
THE UNIVERCITY OF YAOUNDE 1
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING OF YAOUNDE
****
AN-ING1

ANI-2083 - MODELE DE LA SAT ET IA DANS LES ARTS TECHNOLOGIQUES

RAPPORT : AI Sketch - Outil intelligent de dessin et reconnaissance d'objets

Membres du groupe	Matricule
Nyeck Abondo Emile Maël	25P938
NGASSAM BODIONG ANGE KYLIAN	
ZEKENG NGUIMKIA GRACE DIVINE	
NGAMALEU MONKAM RUSSELLE	
TAPA FOTSO PASQUIN	

Sous la haute supervision de :

Dr. Aurelle TCHAGNA

Table des matières

1 Présentation du projet 3

1.1 Technologies utilisées 3

2 Architecture du projet 4

2.1 Modules d'ingestion et de Normalisation 4

3 Résultats Expérimentaux 4

4 Architecture UI, Intégration 5

4.1 Structure du code source 5

4.2 Algorithme de Prétraitement et d'Inférence Temps Réel 5

1 Présentation du projet

Le projet AI-Sketch tel que notre groupe l'a conçu, est une application Desktop interactive permettant le dessin et l'identification en temps réel des croquis dessinés à la main par l'utilisateur. L'application s'appuie sur un réseau de neurones convolutifs CNN développé sous TensorFlow 2.17, et entrainé sur 720000 échantillons issus du jeu de données open-source Quick, Draw! de Google. L'interface graphique sous PyQt6 offre un canevas de dessin vectoriel interactif, un pipeline de prétraitement automatique (redimensionnement 28x28, normalisation et inversion intelligente du contraste) et un affichage dynamique du Top-3 des prédictions. Sur un ensemble de test indépendant de 28 000 images, le modèle atteint une précision globale de 97,65 %.

1.1 Technologies utilisées

Pour ce projet, nous sommes partis sur un ensemble de 6 bibliothèques, représentant les prérequis à la construction de notre application par un tiers :

Composant	Version	Rôle dans le projet
Python	3.12.3	Interprète et langage de développement principal
TensorFlow	2.17.0	Construction, entraînement et inférence du réseau CNN
NumPy	1.26.4	Manipulation matricielle optimisée des tenseurs d'images
PyQt6	6.x	Framework GUI pour l'interface utilisateur interactive
OpenCV	4.	Traitement de bas niveau et gestion d'images sans dépendances GUI
Scikit-Learn	1.x	Calcul des métriques (rapport de classification, matrice de confusion)

2 Architecture du projet

2.1 Modules d'ingestion et de Normalisation

Les données d'apprentissage brutes proviennent de fichiers binaires .npy stockés dans data/raw/. Afin de faire le pont entre les libellés en français et les noms de fichiers Quick, Draw! officiels, un dictionnaire de correspondance FR_TO_EN a été intégré :

```
FR_TO_EN = {
    "pomme": "apple",
    "couronne": "crown",
    "diamant": "diamond",
    "porte": "door",
    "avion": "full_numpy_bitmap_airplane",
    "chat": "full_numpy_bitmap_cat",
    "marteau": "hammer",
}
```

La fonction load_data() extrait 90 000 échantillons par classe (140 000 au total), normalise les intensités de pixels de [0, 255] vers [0.0, 1.0] et façonne le tenseur sous la forme (N, 28, 28, 1). La répartition train/test s'effectue avec un ratio 80/20 stratifié.

3 Résultats Expérimentaux

L'entraînement utilise l'optimiseur Adam et la fonction de perte sparse_categorical_crossentropy. Un callback EarlyStopping avec patience=2 est configuré pour surveiller la perte de validation (val_loss) et restaurer automatiquement les meilleurs poids (restore_best_weights=True).

Époque	Loss (Train)	Accuracy (Train)	Loss (Val)	Accuracy (Val)	Remarque
Époque 1	0.3152	90.33 %	0.1874	93.96 %	Amélioration
Époque 2	0.1820	94.47 %	0.1461	95.70 %	Amélioration
Époque 3	0.1457	95.45 %	0.1367	95.95 %	Amélioration
Époque 4	0.1229	96.23 %	0.1275	96.21 %	Amélioration

Époque 5	0.1056	96.62 %	0.1285	96.23 %	Stagnation
Époque 6	**0.0897**	**97.16 %**	**0.1258**	**96.38 %**	**Optimum atteint (Best)**
Époque 7	0.0762	97.40 %	0.1332	96.21 %	Patience (1/2)
Époque 8	0.0647	97.83 %	0.1326	96.30 %	Arrêt précoce (2/2)

4 Architecture UI, Intégration

4.1 Structure du code source

Arborescence logicielle du projet

```
AI-Sketch-G2/
├── models/
│   └── sketch_classifier.keras       # Modèle pré-entraîné au format Keras
├── output/
│   ├── courbes_entrainement.png      # Graphique des époques (Loss / Accuracy)
│   ├── matrice_confusion.png         # Matrice de confusion
│   └── rapport_classification.txt    # Rapport textuel scikit-learn
├── main.py                           # Point d'entrée de l'application
└── src/
    ├── Trainning/
    │   ├── config.py                 # Configuration centralisée (CLASSES, EPOCHS)
    │   └── train.py                  # Script d'entraînement automatisé
    └── UI/
        ├── main_window.py            # Contrôleur principal et intégration IA
        ├── drawing.py                # Canevas de dessin (DrawingCanvas) & export 28x28
        ├── reconnaissance.py         # Panneau de résultats (RecognitionPanel)
        ├── interface.py              # Composant carte de proposition (SuggestionCard)
        ├── action.py                 # Boutons d'action (Reconnaître, Effacer)
        └── utils.py                  # Normalisation vectorielle et inversion de couleur
```

4.2 Algorithme de Prétraitement et d'Inférence Temps Réel

Lors du clic sur 'Reconnaître', le canevas convertit les traits vectoriels en une QImage 28x28. La fonction qimage_to_tensor() assure l'inversion automatique si le fond est blanc (valeur moyenne > 127) afin d'aligner parfaitement le dessin utilisateur sur le format Quick, Draw! (fond noir = 0.0, traits blancs = 1.0).